

Prompt Iteration Log (6 Runs Total)

Run	Technique	Observed Output Improvement & Notes
Run 0	Naive Baseline	1-line code fix suggestion; ignored tensor semantics and pipeline data leakage.
Run 1	Role Assignment	Minimal Value: Added intro filler ('As a Senior ML Engineer...') without extra code context.
Run 2	Context & Motivation	Major Jump: Pinpointed exact error line AND proactively caught upstream scaler leakage.
Run 3	Few-Shot Examples	Adhered to demonstrated diagnostic breakdown (Problem -> Cause -> Fix -> Leakage Audit).
Run 4	Output Structure	Separated fix into 4 Markdown sections (Root Cause, Line Fix, Data Audit, Refactored Code).
Run 5	Step Decomposition	Uncovered 'BCEWithLogitsLoss' log-sum-exp numerical stability fix and added runtime assertions.

Cross-Model Comparison: Claude vs ChatGPT (GPT-4o)

Dimension	Claude 3.5 Sonnet	ChatGPT GPT-4o	Winner & Key Difference
Technical Accuracy	Fixed shape, caught leakage, recommended BCEWithLogitsLoss for log-sum-exp stability.	Fixed shape and caught leakage; kept BCELoss + Sigmoid.	Claude (Identified deeper numerical stability optimization).
Formatting	100% section adherence; zero intro filler.	Followed sections, added intro filler.	Claude (Cleaner constraint adherence).
Code Quality	Full runnable code with explicit `assert` statements.	Full runnable code, omitted runtime assertions.	Claude (More defensive production code).

Final Reusable Debugging Template

[ROLE] Senior ML Engineer & Code Reviewer. [CONTEXT] Debugging Python/ML issue in production. [INPUT] Error Traceback: [TRACEBACK] | Code Snippet: [CODE] | Expected: [BEHAVIOR] [STEPS] 1. Inspect shapes/types. 2. Audit pipeline leakage. 3. Root cause & stability. 4. Refactored code. [FORMAT] 4 Sections: 1. Root Cause 2. Immediate Fix 3. Data Audit 4. Refactored Code with Assertions. [CONSTRAINTS] Never swallow exceptions; no conversational filler; include assertions.